

APB Master Verification Report

Overall Code Coverage: 85.55%

Overview

A SystemVerilog class-based verification environment was developed to verify the APB Master. The environment consists of a generator, driver, monitor, reference model, scoreboard, environment, interface and top-level testbench. Directed and constrained-random tests were executed to verify APB protocol functionality.

DUT Bug Identified

Bug 1: Incorrect FSM Transition for Back-to-Back Transfers

- The FSM always transitions from the **ACCESS** state to the **IDLE** state when PREADY is asserted.
- According to the APB protocol, if a new transfer request (transfer=1) is pending when the current transfer completes, the FSM should transition directly from **ACCESS** to **SETUP** instead of returning to **IDLE**.
- This implementation introduces an unnecessary idle cycle and prevents efficient back-to-back transfers.

Bug 2: One Clock Cycle Delay in APB Output Signals

- The APB output signals are generated inside a sequential (posedge) block based on the current FSM state.
- As a result, these signals become valid one clock cycle later than expected.
- This additional clock delay causes the monitor to observe the transaction outputs one cycle late, resulting in incorrect APB timing.

Bug 3: Delayed Assertion of PWRITE

- The PWRITE signal is updated only when the FSM enters the **SETUP** state.
- Consequently, PWRITE is asserted one clock cycle later than required.
- According to the APB protocol, PWRITE must be valid during the setup phase along with PSEL, PADDR, and the remaining control signals so that the slave correctly identifies the transaction type.

Bug 4: Input Signals Not Captured for Back-to-Back Transfers

- The transaction inputs (addr_in, wdata_in, strb_in, write_read) are captured only when the FSM is in the **IDLE** state.
- If a new transfer request is asserted while the current transaction is completing (ACCESS state with PREADY=1 and transfer=1), the new input values are not sampled.

- As a result, even after correcting the FSM for back-to-back transfers, the subsequent transaction may use stale address, data, and control information. The input capture logic should also support loading new transaction information when a back-to-back transfer is initiated.

Coverage Notes / Justification for Uncovered Items

- Overall code coverage achieved is 85.55%.

Questa Branch Coverage Report			
Show All Show Covered Show Missing			
if(exp_trans.write_read==1 && act_trans.write_read==1) begin			66.66%
Branch	Source	Hits	Status
IF	if(exp_trans.write_read==1 && act_trans.write_read==1) begin	23	Covered
TRUE	else if(act_trans.write_read==0 && exp_trans.write_read==0) begin	7	Covered
ELSE	else begin	0	ZERO
if((exp_trans.PSEL==act_trans.PSEL) && (exp_trans.PENABLE== act_trans.PEN...			50.00%
Branch	Source	Hits	Status
IF	if((exp_trans.PSEL==act_trans.PSEL) && (exp_trans.PENABLE== act_trans.PEN...	23	Covered
ELSE	else begin	0	ZERO
if((act_trans.PSEL==exp_trans.PSEL) && (act_trans.PENABLE==exp_trans.PENA...			50.00%
Branch	Source	Hits	Status
IF	if((act_trans.PSEL==exp_trans.PSEL) && (act_trans.PENABLE==exp_trans.PENA...	7	Covered
ELSE	else begin	0	ZERO

- Scoreboard coverage is lower because the uncovered branches correspond to error-handling (else) paths. During normal verification, the DUT output matched the reference model, so mismatch branches were intentionally not executed.
- Some scoreboard branches are only exercised when expected and actual transactions differ. Since no mismatches occurred during normal tests, these branches remain uncovered.

Questa Condition Coverage Report			
Show All Show Covered Show Missing			
FEC Condition: (vif.mon_cb.PSEL && vif.mon_cb.PENABLE && ~vif.mon_cb.PREADY)			33.33%
Input Term	Covered	Reason For No Coverage	Hint
vif.mon_cb.PSEL	No	'_0' not hit	Hit '_0'
vif.mon_cb.PENABLE	No	'_0' not hit	Hit '_0'
vif.mon_cb.PREADY	Yes		
Rows	FEC Target	Hits	Non-Masking Condition(s)
Row 1	vif.mon_cb.PSEL_0	0	-
Row 2	vif.mon_cb.PSEL_1	1	(vif.mon_cb.PENABLE && ~vif.mon_cb.PREADY)
Row 3	vif.mon_cb.PENABLE_0	0	vif.mon_cb.PSEL
Row 4	vif.mon_cb.PENABLE_1	1	(vif.mon_cb.PSEL && ~vif.mon_cb.PREADY)
Row 5	vif.mon_cb.PREADY_0	1	(vif.mon_cb.PSEL && vif.mon_cb.PENABLE)
Row 6	vif.mon_cb.PREADY_1	1	(vif.mon_cb.PSEL && vif.mon_cb.PENABLE)

- In the monitor, one FEC condition is not obtainable because sampling occurs only after waiting for both PSEL and PENABLE. At that point the protocol is already in the ACCESS state, and the monitor counts wait cycles until PREADY becomes high.

Scope: [/apb_pkg/apb_driver](#)
Covergroup type: [drv_cg](#)

Cross: wr_strb

Crossed Coverpoints : wr_rd , strb

Search:

wr_rd	strb	At Least	Hits
read	all_ones_strb	1	0
read	single_bit_strb	1	0
write	zero_strb	1	3
read	zero_strb	1	7
write	all_ones_strb	1	2
write	single_bit_strb	1	4

- A cross-coverage bin involving read/write and PSTRB cannot be hit because APB read transactions do not use write strobes. The verification environment also constrains read transactions so that PSTRB is not asserted.

Scope: [/apb_pkg/apb_monitor](#)
Covergroup type: [mon_cg](#)

Cross: wr_str

Crossed Coverpoints : wr_rd , str

Search:

Bin Name	At Least	Hits
write_strobe	1	7

Search:

wr_rd	str	At Least	Hits
read	ones_strb1	1	0
read	zero_strb1	1	7

- Similarly, monitor bins involving read transactions with valid strobe values cannot be covered because such protocol combinations are illegal.

case (state)			75.00%
Branch	Source	Hits	Status
TRUE	IDLE: begin	50	Covered
TRUE	SETUP: begin	47	Covered
TRUE	ACCESS: begin	48	Covered
TRUE	default: next_state = IDLE;	0	ZERO

if (transfer)			100.00%
Branch	Source	Hits	Status
IF	if (transfer)	48	Covered

- In the DUT, one branch cannot be covered because the FSM state is encoded as a 2-bit signal and for 00, 01, 11 cases explicitly handled in the case statement. There is no case where it goes to 10 case, therefore that particular branch is not exercised.

Test Cases Executed

- Reset
- Single Write
- Single Read
- Back-to-Back Transfers
- Random Read/Write
- Random Address

Coverage Screenshots

Questa Coverage Report

Number of tests run:	1
Passed:	0
Warning:	1
Error:	0
Fatal:	0

[List of tests included in report...](#)

[List of global attributes included in report...](#)

[List of Design Units included in report...](#)

Coverage Summary by Structure:

Design Scope ◀	Hits % ◀	Coverage % ◀
top	98.98%	96.19%
intrf	99.14%	98.12%
DUV	99.03%	96.34%
apb_pkg	82.74%	65.84%
apb_transaction/copy	100.00%	100.00%
apb_generator/new	100.00%	100.00%
apb_generator/start	100.00%	100.00%
apb_driver	74.46%	81.26%
apb_monitor	94.23%	76.19%
apb_scoreboard/new	100.00%	100.00%
apb_scoreboard/start	100.00%	100.00%
apb_scoreboard/compare_report	28.94%	33.76%
apb_environment/new	100.00%	100.00%
apb_environment/build	100.00%	100.00%
apb_environment/start	100.00%	100.00%
apb_test/new	100.00%	100.00%
apb_test/run	100.00%	100.00%

Coverage Summary by Type:

Total Coverage:					94.76%	85.55%
Coverage Type ◀	Bins ◀	Hits ◀	Misses ◀	Weight ◀	% Hit ◀	Coverage ◀
Covergroups	42	39	3	1	92.85%	95.95%
Statements	308	276	32	1	89.61%	89.61%
Branches	44	39	5	1	88.63%	88.63%
FEC Conditions	28	12	16	1	42.85%	42.85%
Toggles	754	749	5	1	99.33%	99.33%
FSMs	7	6	1	1	85.71%	87.50%
States	3	3	0	1	100.00%	100.00%
Transitions	4	3	1	1	75.00%	75.00%
Assertions	20	19	1	1	95.00%	95.00%

Report generated by [Questa](#) (ver. 10.6c) on Sun 19 Jul 2026 12:45:12 PM IST with command line:
vcover report -html apb_coverage.ucdb -htmldir covReport -details